

Proyecto de Curso: Análisis, Diseño y Construcción de un Sistema Operativo desde Cero

Por

Castro Pari, Rayneld Fidel

Mamani Flores, Natan

Mendoza Quispe, Jose Daniel

Polo Chura, Marco Rosauro

Trabajo académico presentado a la

Facultad de Ingeniería Eléctrica, Electrónica, Informática y Mecánica

como

Proyecto de Investigación de la Primera Unidad de Sistemas Operativos



Departamento Académico de Ingeniería de Informática y de Sistemas

Asesor: Ugarte Rojas, Héctor Eduardo

Memorial Universidad Nacional San Antonio Abad del Cusco

Semestre 2025-II

Cusco

Perú

Índice general

Índice de tablas	iv
Índice de figuras	v
Introducción	1
1 Propuestas analizadas	4
1.1 RedoxOS	5
1.1.1 Nombre del proyecto o sistema operativo	5
1.1.2 Enlace al repositorio y/o documentación oficial	5
1.1.3 Objetivo del proyecto	5
1.1.4 Lenguaje(s) de implementación	6
1.1.5 Arquitectura del sistema	6
1.1.6 Componentes implementados	7
1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.) . . .	10
1.1.8 Nivel de complejidad y accesibilidad para estudiantes	10
1.2 FlexOS	11
1.2.1 Nombre del proyecto o sistema operativo	11

1.2.2	Enlace al repositorio y/o documentación oficial	11
1.2.3	Objetivo del proyecto	12
1.2.4	Lenguaje(s) de implementación	12
1.2.5	Arquitectura del sistema	12
1.2.6	Componentes implementados	12
1.2.7	Herramientas utilizadas (compiladores, emuladores, etc.) . . .	12
1.2.8	Nivel de complejidad y accesibilidad para estudiantes	12
2	Comparación técnica	13
3	Análisis crítico	14
	Referencias	15

Índice de tablas

1.1	Funcionamiento técnico (idea clave) de los principales subsistemas de	
	Redox OS	8

Índice de figuras

1.1	Comparativa de la arquitectura de Redox frente a arquitecturas tradicionales.	7
-----	---	---

Introduccion

El desarrollo y la transformación de los sistemas operativos han sido fundamentales en la evolución de las tecnologías de la información. Desde los primeros programas creados para las computadoras centrales hasta los sistemas más utilizados en ordenadores personales, dispositivos móviles y equipos embebidos, los sistemas operativos han actuado como un puente entre las necesidades del usuario y la complejidad del hardware. Estudiarlos no solo permite entender el funcionamiento básico de una computadora, sino que también brinda las bases conceptuales y técnicas necesarias para el diseño de nuevas soluciones informáticas. Este documento tiene como finalidad ofrecer una perspectiva global de los sistemas operativos, abordando sus características principales, su arquitectura y sus componentes.

Objetivo de la investigacion

El objetivo principal de este trabajo es realizar un análisis técnico y comparativo de distintos sistemas operativos, con la finalidad de identificar sus características más relevantes y examinar las diversas concepciones y enfoques aplicados en su diseño e implementación. Se estudiarán tanto arquitecturas tradicionales, como la monolítica,

así como modelos más modernos, entre ellos los exokernels, híbridos y microkernels . Del mismo modo, se pretende ofrecer una visión crítica sobre las ventajas, limitaciones y ámbitos de aplicación de cada sistema operativo.

Alcance del análisis

El presente estudio se centra en sistemas operativos ampliamente utilizados y representativos de diferentes enfoques arquitectónicos y áreas de aplicación. No se abordarán sistemas altamente experimentales o versiones obsoletas a menos que sean relevantes para la comparación histórica o conceptual. El análisis incluye tanto sistemas de propósito general (Windows, macOS, Linux) como sistemas especializados (RTOS, embebidos y móviles), enfocándose en aspectos técnicos, operativos y comunitarios que permitan identificar fortalezas, limitaciones y escenarios de uso recomendados para cada sistema operativo.

Metodologia

Con el fin de alcanzar los objetivos planteados, se seguirá una metodología compuesta por tres fases fundamentales:

1. **Revisión bibliográfica y documental:** Se consultarán libros especializados, artículos académicos, blogs técnicos, documentación oficial y repositorios relacionados con los sistemas operativos en estudio.
2. **Evaluación técnica:** Se examinarán los aspectos esenciales de cada sistema, incluyendo la gestión de procesos, la administración de memoria, los sistemas de

archivos, el manejo de dispositivos, las interfaces de usuario y los mecanismos de seguridad.

3. **Comparación estructurada:** Se elaborará una tabla que sintetice las características más representativas de los sistemas operativos analizados, tomando en cuenta factores como el tamaño del kernel, la arquitectura, la comunidad, el lenguaje de implementación, la documentación disponible y el soporte de hardware.

Este enfoque permitirá garantizar un análisis más objetivo, completo y útil, tanto para fines académicos como prácticos.

Capítulo 1

Propuestas analizadas

1.1 RedoxOS

1.1.1 Nombre del proyecto o sistema operativo

El sistema operativo examinado es "Redox OS". De acuerdo con (Redox OS Project, 2025a), su denominación deriva de la palabra "Redox", que es la abreviatura de "*reduction-oxidation*", un término de química que representa el intercambio equilibrado de electrones; esto refleja la meta del proyecto: alcanzar un sistema equilibrado en seguridad, rendimiento y simplicidad, a través de un diseño contemporáneo y fiable, redactado totalmente en Rust.

1.1.2 Enlace al repositorio y/o documentación oficial

- Enlace al repositorio oficial: <https://github.com/redox-os>
- Enlace al libro oficial: <https://doc.redox-os.org/book/>

1.1.3 Objetivo del proyecto

Según (Redox OS Project, 2025a,b), Redox OS tiene un enfoque fundamentalmente educativo y experimental, pero también pretende ofrecer una opción práctica y segura en comparación con sistemas Unix tradicionales. Su objetivo es probar que un sistema operativo que esté totalmente desarrollado en Rust puede lograr la misma estabilidad y eficiencia que los sistemas comerciales, mientras elimina categorías completas de errores relacionados con la memoria y la concurrencia.

1.1.4 Lenguaje(s) de implementación

Redox OS está predominantemente desarrollado en Rust, abarcando su kernel, controladores, gestor de memoria, sistema de archivos y gran parte de las herramientas de usuario.

Ciertos elementos específicos, como el cargador de arranque y aspectos de la compatibilidad con C, emplean "ensamblador y C" de manera restringida.

1.1.5 Arquitectura del sistema

Redox OS adopta una arquitectura híbrida de microkernel, inspirada en el diseño de Minix 3 y el modelo de Unix, pero con un enfoque mucho más fuerte en la seguridad y la modularidad. El kernel de Redox es extremadamente pequeño: se encarga solo de la planificación, la gestión básica de memoria y la comunicación entre procesos. Todos los servicios del sistema, como el manejo de archivos, controladores, red y entorno gráfico, se ejecutan en el espacio de usuario como procesos independientes llamados schemes.

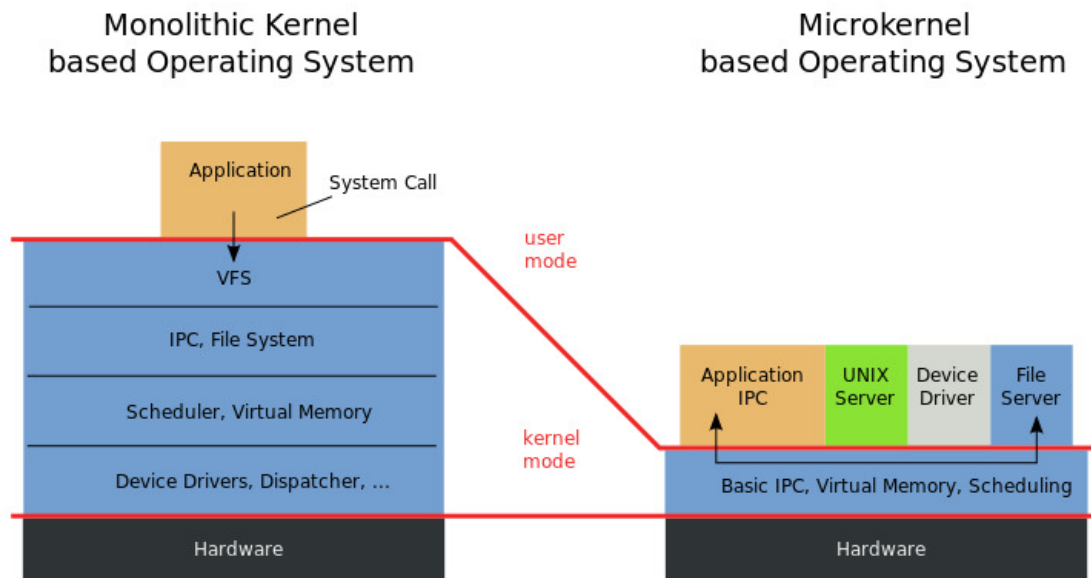


Figura 1.1: Comparativa de la arquitectura de Redox frente a arquitecturas tradicionales.

Fuente: Obtenido de (Redox OS Project, 2025a).

1.1.6 Componentes implementados

En Redox OS, los componentes principales del sistema se implementan como crates escritos en Rust, siguiendo una arquitectura microkernel. Cada crate encapsula un servicio o subsistema (procesos, memoria, sistema de archivos, E/S, etc.) con una estricta separación de privilegios. El microkernel se encarga de la comunicación por mensajes entre estos servicios, garantizando seguridad, aislamiento y estabilidad.

A continuación, se presenta una tabla con los principales componentes y subsistemas de Redox OS, los nombres de sus crates en el repositorio oficial y la idea técnica detrás de su funcionamiento.

Tabla 1.1: Funcionamiento técnico (idea clave) de los principales subsistemas de Redox OS

Componente / Sub-sistema	Nombre en Redox (archivo/crate)	Funcionamiento técnico (idea clave)
Gestión de memoria	<code>memory, paging, rmm</code>	Implementa paginación virtual por demanda y gestión de marcos físicos.
Gestión de procesos y multitarea	<code>kernel, scheme::proc</code>	Cada proceso se ejecuta en espacio de usuario, gestionado por el microkernel.
Sistema de archivos	<code>redoxfs</code>	Sistema de archivos nativo de Redox, implementado completamente en Rust.
Gestión de dispositivos y E/S	<code>drivers, schemes</code>	Controladores modulares gestionados como servicios de usuario <code>schemes</code> .
Comunicación entre procesos (IPC)	<code>syscall, scheme</code>	Utiliza el modelo <i>scheme</i> para llamadas al sistema y paso de mensajes entre procesos y servicios del kernel.
Planificación y temporización	<code>scheduler, timer</code>	Planificador cooperativo basado en colas de prioridad.
Gestión de usuarios y permisos	<code>user, authd</code>	Módulos dedicados al control de autenticación y separación de privilegios mediante el servicio <code>authd</code> .
Interfaz gráfica (GUI)	<code>orbital, orbtok</code>	Entorno gráfico modular escrito

Gestión de memoria

La gestión de memoria es fundamental para el kernel de cualquier sistema operativo . Proporcionar una forma rápida de que los programas asignen y liberen memoria regularmente es una responsabilidad fundamental del kernel. Existen diversas implementaciones para asignar memoria física , como mapas de bits, asignación de amigos y el uso de estructuras de árbol o colas/pilas (OSDev Wiki, 2025).

Scheduling / multitarea

Un scheduling algorithm determina cuánto tiempo de CPU se asigna a los procesos y subprocesos. El objetivo de cualquier algoritmo de planificación es cumplir con una serie de criterios fundamentales:

- **Equidad:** ninguna tarea debe quedarse sin recursos; todas deben tener la oportunidad de usar tiempo de CPU.
- **Prioridad:** si se emplean niveles de prioridad, una tarea de baja prioridad no debe retrasar la ejecución de una tarea de alta prioridad.
- **Escalabilidad:** el planificador debe mantener un rendimiento estable ante un número creciente de tareas, idealmente con una complejidad de $O(1)$, como ocurre en el kernel de Linux.

Subsistemas de E/S y drivers

Comunicación entre módulos

1.1.7 Herramientas utilizadas (compiladores, emuladores, etc.)

1.1.8 Nivel de complejidad y accesibilidad para estudiantes

1.2 FlexOS

1.2.1 Nombre del proyecto o sistema operativo

El sistema operativo examinado es "Redox OS". De acuerdo con (Redox OS Project, 2025a), su denominación deriva de la palabra "Redox", que es la abreviatura de "*reduction-oxidation*", un término de química que representa el intercambio equilibrado de electrones; esto refleja la meta del proyecto: alcanzar un sistema equilibrado en seguridad, rendimiento y simplicidad, a través de un diseño contemporáneo y fiable, redactado totalmente en Rust.

1.2.2 Enlace al repositorio y/o documentación oficial

- **Enlace al repositorio oficial:** <https://github.com/redox-os>
- **Enlace al libro oficial:** <https://doc.redox-os.org/book/>

1.2.3 Objetivo del proyecto

1.2.4 Lenguaje(s) de implementación

1.2.5 Arquitectura del sistema

1.2.6 Componentes implementados

Gestión de memoria

Carga dinámica de módulos (módulos / células)

Scheduling / multitarea

Subsistemas de E/S y drivers

Recuperación ante fallos / actualización en caliente

Comunicación entre módulos

1.2.7 Herramientas utilizadas (compiladores, emuladores, etc.)

1.2.8 Nivel de complejidad y accesibilidad para estudiantes

Capítulo 2

Comparación técnica

La tabla siguiente presenta una comparación técnica de varios sistemas operativos destacados, considerando aspectos clave como arquitectura, lenguaje de programación, tamaño del kernel, soporte de hardware, documentación y comunidad de usuarios y desarrolladores.

Capítulo 3

Análisis crítico

Referencias

OSDev Wiki (2025). Memory Management – OSDev Wiki. Accedido: 25 octubre 2025.

Redox OS Project (2025a). Documentación oficial de Redox OS. Accedido: 22 octubre 2025.

Redox OS Project (2025b). Repositorio de Redox OS en GitHub. Accedido: 22 octubre 2025.